

Web Vulnerability Assessment Report

1. Executive Summary

This report presents a detailed analysis of vulnerabilities identified in the DVWA environment. The assessment focused on identifying common web security issues such as SQL Injection and Cross-Site Scripting (XSS). Testing was performed in a controlled lab environment using industry-standard tools.

2. Tools & Environment

- Burp Suite – for intercepting HTTP requests
- DVWA – vulnerable web application for testing
- Web Browser – for interaction
- Kali Linux – testing platform

3. Vulnerability Testing

3.1 SQL Injection

Procedure

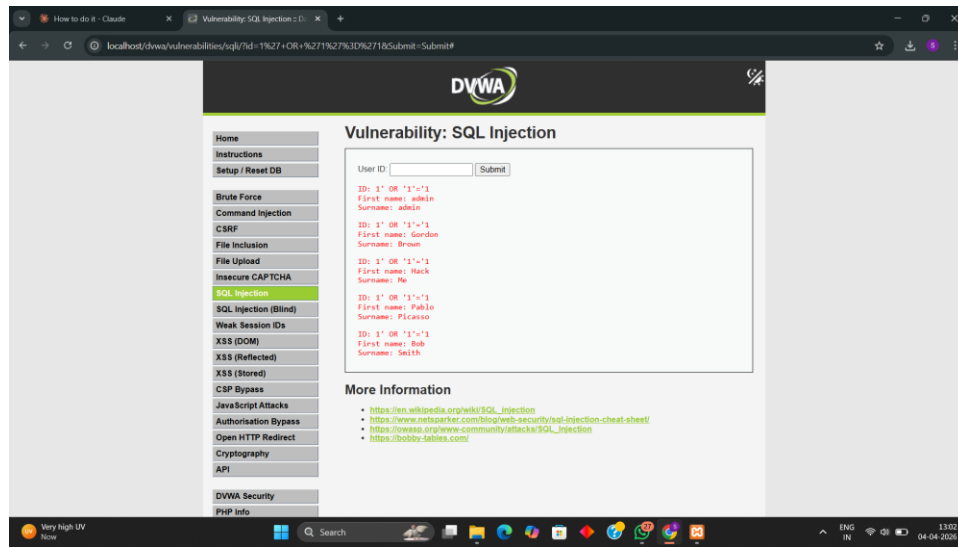
1. Open DVWA and navigate to the SQL Injection module
2. Enter payload: ' OR '1'='1 into the input field
3. Submit the request
4. Observe the application response

Findings

- The application failed to validate user input
- Database query was manipulated successfully
- All records were displayed instead of filtered results
- This confirms vulnerability to SQL Injection

SQL Injection is a critical vulnerability that allows attackers to manipulate database queries. If exploited, it can lead to unauthorized data access, data modification, or complete database compromise.

Result



3.2 Cross-Site Scripting (XSS)

Procedure

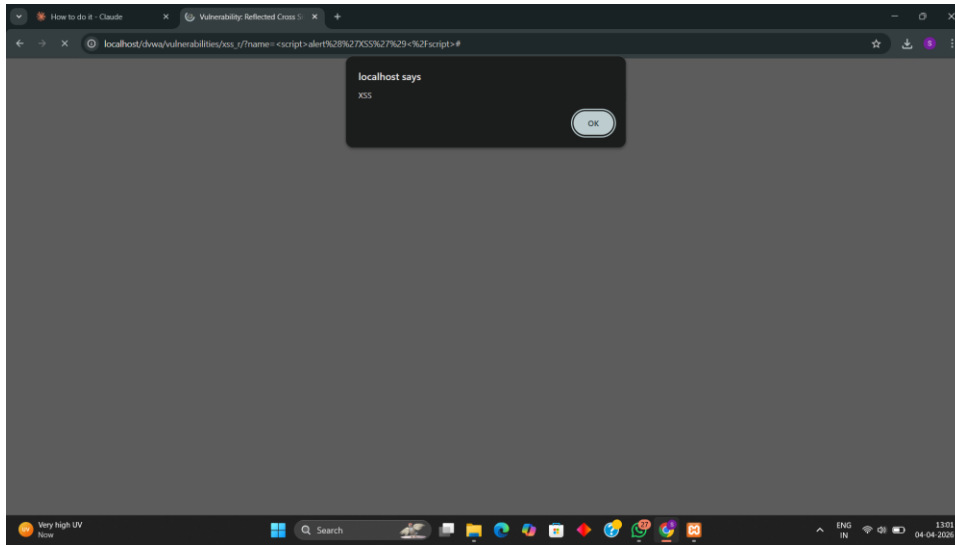
5. Navigate to the XSS module in DVWA
6. Enter payload: `<script>alert('XSS')</script>`
7. Submit the input
8. Observe the alert popup

Findings

- The application accepted unsanitized input
- JavaScript executed in browser context
- No output encoding was applied
- This confirms XSS vulnerability

Cross-Site Scripting (XSS) allows attackers to inject malicious scripts into web pages viewed by other users. This can result in session hijacking, credential theft, and malicious redirections.

Result



3.3 Burp Suite Interception

Procedure

9. Configure browser proxy settings to route traffic through Burp Suite
10. Enable intercept mode in Burp Suite
11. Perform actions in DVWA
12. Capture and analyze HTTP requests

Findings

- HTTP requests and responses were successfully intercepted
- Sensitive parameters were visible in plain text
- Session cookies could be observed
- This indicates risk of session hijacking

Burp Suite demonstrated how attackers can intercept and analyze web traffic. If communication is not properly secured, attackers can exploit this to steal sensitive data or manipulate requests.

Result

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response
1	https://localhost	GET	/dewa			301	412	HTML		301 Moved Permanently			127.0.0.1		13:27:02.4 A...	8080	75
2	https://localhost	GET	/dewa/			302	532	HTML					127.0.0.1		13:27:03.4 A...	8080	218
3	https://www.google.com	GET	/search/warmup.html			200	1799	HTML	html	Warmup Page		✓	142.251.156.119	security=low PH...	13:27:03.4 A...	8080	81
4	https://localhost	GET	/dewa/login.php			200	1096	HTML	php	Login - Daman Vulnera...		✓	127.0.0.1	__Secure-STRP=A...	13:27:03.4 A...	8080	199
8	https://github.com	GET	/diganinja/DVWA/			200	582181	HTML		GitHub - diganinja/DV...		✓	20.207.73.82	_gh_x=1=9m219...	13:27:35.4 A...	8080	912
37	https://github.com	GET	/manifest.json			200	5631	JSON	json			✓	20.207.73.82		13:27:39.4 A...	8080	31
79	https://github.com	GET	/diganinja/DVWA/sponsor_button			200	8528	HTML				✓	20.207.73.82		13:27:40.4 A...	8080	383
80	https://github.com	GET	/diganinja/DVWA/contributor_list...		✓	200	11623	HTML				✓	20.207.73.82		13:27:40.4 A...	8080	339
81	https://github.com	GET	/diganinja/DVWA/nonce_by_id		✓	200	4080	text				✓	20.207.73.82		13:27:40.4 A...	8080	382
83	https://github.com	GET	/diganinja/DVWA/background_suffi...		✓	200	5686	HTML				✓	20.207.73.82		13:27:40.4 A...	8080	400
84	https://github.com	GET	/diganinja/DVWA/hovercards/ctati...		✓	204	4037					✓	20.207.73.82		13:27:40.4 A...	8080	362
86	https://github.com	GET	/diganinja/DVWA/sponsors_list/...		✓	200	6311	HTML				✓	20.207.73.82		13:27:40.4 A...	8080	343

4. Conclusion

The assessment identified critical vulnerabilities including SQL Injection and Cross-Site Scripting. These issues arise due to improper input validation and lack of secure coding practices. It is recommended to implement parameterized queries, input sanitization, and secure session handling mechanisms.